

LLM-ETF：

基於大語言模型輔助之跨市場 ETF 趨勢預測與模型比較研究

Graduate School of Information Engineering
Kun Shan University

研究生：林子瑜

Tzu-Yu Lin

指導教授：李冠榮 副教授

Kuan-Rong Lee

JUN, 2026

| 簡報大綱 Agenda



研究背景與問題

跨市場投資的痛點、研究問題



跨世代模型介紹

統計、機器學習、深度學習、大語言模型



資料處理與實驗設計

三檔 ETF、技術指標特徵、
時序交叉驗證



實驗結果與跨市場比較

分類與迴歸、MAPE評估



LLM-ETF 平台實作

互動平台、風險摘要



結論與未來方向

研究貢獻與後續延伸

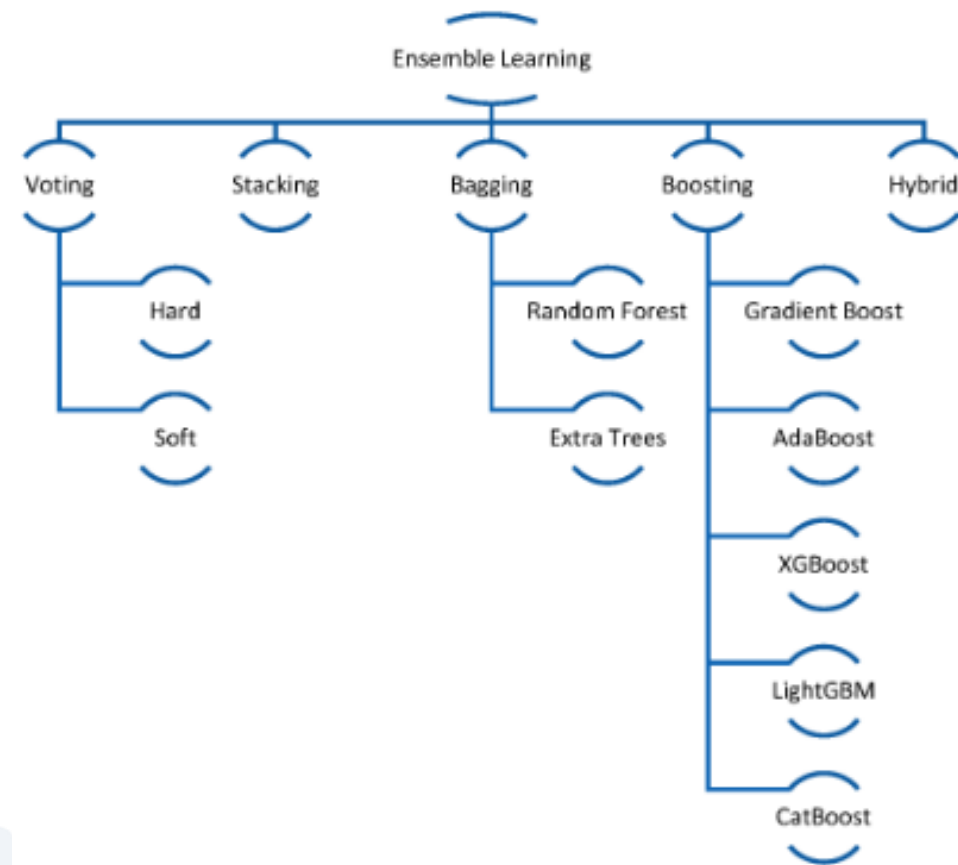
文獻探討(一)：集成學習與台股預測

徐維延 (2019)：TAIEX 趨勢預測研究

- **核心方法**：使用 1999-2018 年共 20 年資料，整合 192 個特徵（含技術面、國際股市與總經指標）。
- **模型**：隨機森林 (RF) 與 AdaBoost 混合模型，採用「增長式視窗滾動法」驗證。
- **關鍵發現**：集成模型受「外推限制」影響，無法預測超出訓練範圍的新高價。
- **關聯**：驗證了本研究中集成模型在歷史高點易出現「預測躺平」的結構性限制。

研究啟發：複雜的混合模型未顯著優於單一隨機森林。

本研究採用 TimeSeriesSplit 確保時序邏輯一致。



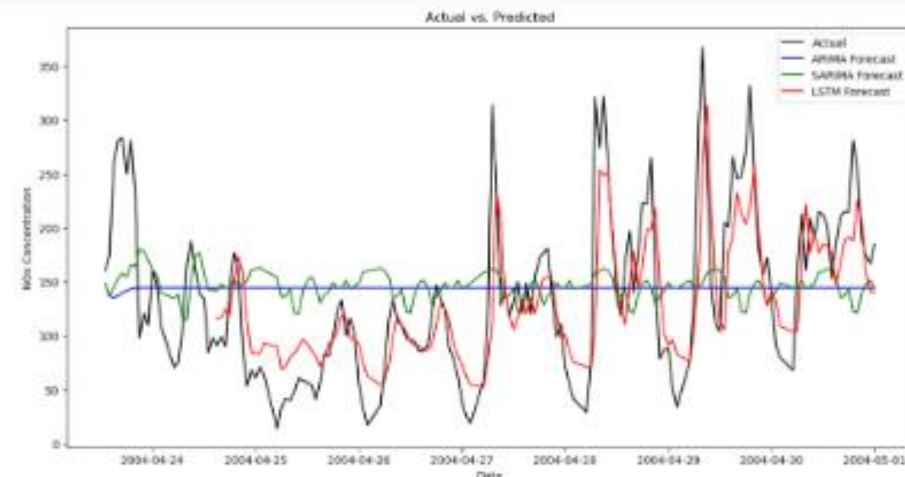
文獻探討(二)：ARIMA 與 LSTM 比較

吳冠霆 (2025)：跨市場指數預測效能比對

- **資料範圍**：涵蓋台、美、日、港主要指數 (2010-2024)。
- **績效對比**：ARIMA 模型在 MAPE 等指標上全面優於深度學習模型 LSTM，且曲線更貼近實際走勢。
- **架構優化**：單層 LSTM (512/1024 單位) 搭配 Sigmoid 函數為最佳配置；增加層數無法提升能力。

準確率優勢	ARIMA (MAPE 0.66-1.07%) > LSTM
運算效率	LSTM 遠快於 ARIMA (後者需頻繁重建模)

關聯：支持本研究中「簡單模型 (ARIMA) 優於複雜模型 (LSTM)」的實驗結果與參數設定。



研究背景與動機 不同模型與技術 == > 股票分析

① 缺乏比較工具

- 市面上缺乏能並排比較台股與美股、又整理得直觀好懂的工具



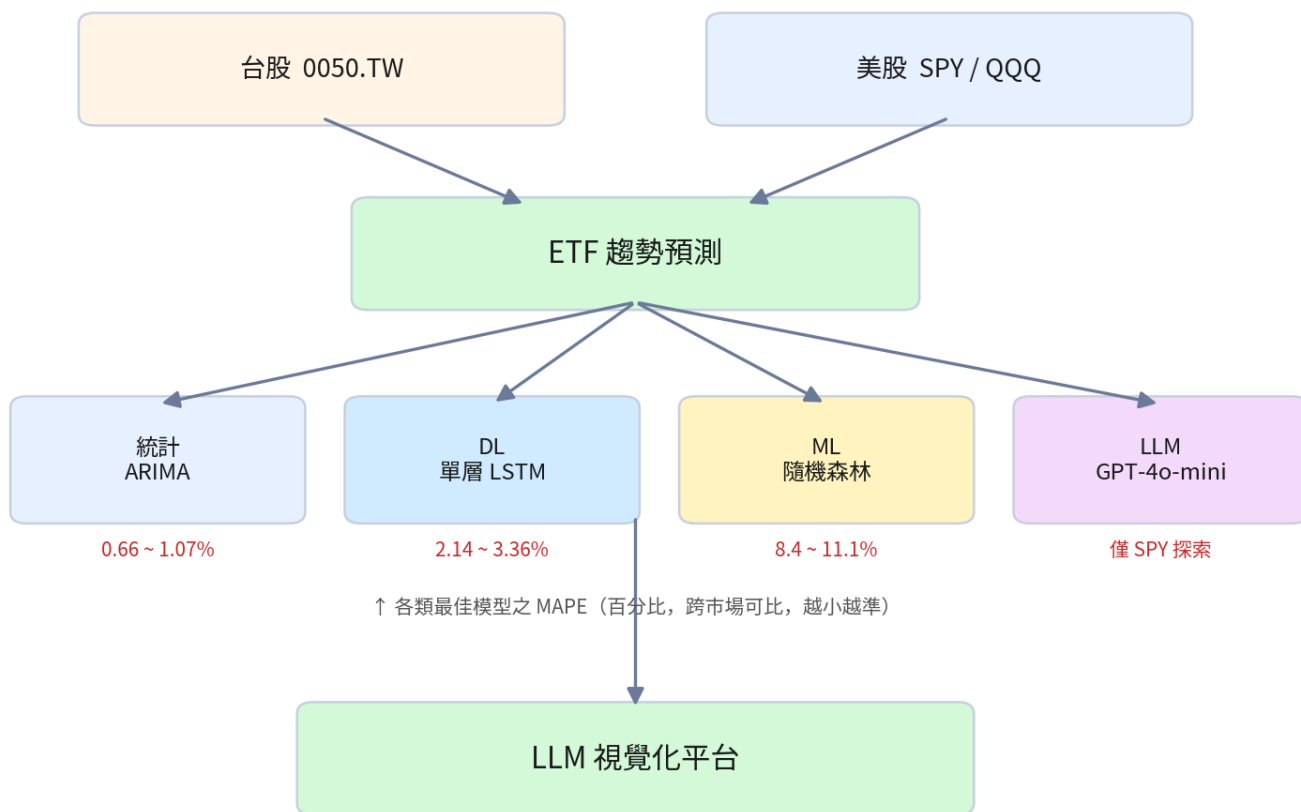
② 模型缺乏系統性比較

- 從統計到深度學習模型眾多,少有研究在同一套資料、同樣任務上公平比較

③ 大語言模型的可能性

- GPT 為最新世代工具,探索其數值預測能力與白話摘要的輔助價值

研究目的與問題



RQ

- RQ1:效能比較：統計（ARIMA）、機器學習（隨機森林等）、深度學習（LSTM）與大語言模型（LLM）在 ETF 預測之優劣
- RQ2:市場差異：分析 台股(0050)與美股 (SPY/QQQ)之預測難度差異
- RQ3:評估指標差異：MAPE（平均絕對百分比誤差）RMSE
- RQ4:實作應用：建構 LLM 輔助視覺化平台，提升預測理解度

分析主題1：分類問題 (Classification)

目標與任務

預測次日收盤價相對於當日是「上漲」或「下跌」，
定義為二元分類任務。

$$y_t = \begin{cases} 1 & \text{if Price}_{t+1} > \text{Price}_t \\ 0 & \text{otherwise} \end{cases}$$

預測目標為「次日」單步預測,以利模型公平比較;
標籤可延伸為一週(t+5)、一個月(t+20)以做中長期預測,
列為後續方向。

AI 預測趨勢

看漲 

↑ R²: 0.92

AI 預測趨勢

看跌 

↑ R²: -2.11

統計模型：ARIMA

為何 $d=1$ ：股價為非定態（有趨勢），須一次差分轉為定態，ARIMA 才能建模； $d=0$ 不差分會殘留趨勢、預測失準。

- ARIMA(自迴歸整合移動平均)是時間序列預測中最常用的統計模型,本研究作為**基準模型(Baseline)**。對單變量時間序列進行建模與預測。

AR

(自迴歸, p)

假設當前值是過去 p 期數值的線性組合。

例:AR(1) 代表今天的值主要受昨天影響。

I

(差分整合, d)

對原始序列做 d 次差分,消除趨勢與單位根,讓序列變成定態

(均值與變異不隨時間漂移)。

這是 ARIMA 相較於 ARMA 的關鍵擴充。

MA

(移動平均, q)

用過去 q 期的預測誤差(殘差)修正當期預測,捕捉短期衝擊的影響。

三市場差分階數一致為 $d=1$ (經 ADF 檢定確認須差分)。

統計模型：ARIMA 核心程式碼

pmdarima : auto_arima 自動選參數 + rolling forecast (arima_baseline.py)

```
49     model = pm.auto_arima(train, start_p=1, start_q=1, max_p=5, max_q=5,  
50                           d=None, seasonal=False, stepwise=True,  
51                           error_action="ignore", suppress_warnings=True)
```

參數說明：

- start_p=1, start_q=1：AR、MA 階數的搜尋起點。
- max_p=5, max_q=5：限制最大階數，避免過度複雜。
- d=None: 差分階數不固定, 由 auto_arima 自動依檢定決定(本研究三市場皆得 d=1)。
- seasonal=False：股價日資料無固定季節週期，關閉季節項（不跑 SARIMA）。
- information_criterion='aic'：以 AIC 在候選模型中選最佳（兼顧配適度與複雜度）。
- stepwise=True：逐步搜尋而非窮舉，速度快且穩定。
- error_action='ignore', suppress_warnings=True：跳過無法配適的組合、抑制警告。

三市場 d 皆為 1, 與 ADF 一致; (p,q) 隨各折變動, 由 auto_arima 自動選定。

機器學習：集成迴歸模型



Random Forest

Bagging 架構，多棵樹平均降低變異數，穩定且不易過擬合。



XGBoost

Boosting 經典模型，含正規化項抑制過擬合，速度快、精度高。



AdaBoost

逐輪提高前一輪誤差較大樣本的權重，串聯弱學習器。

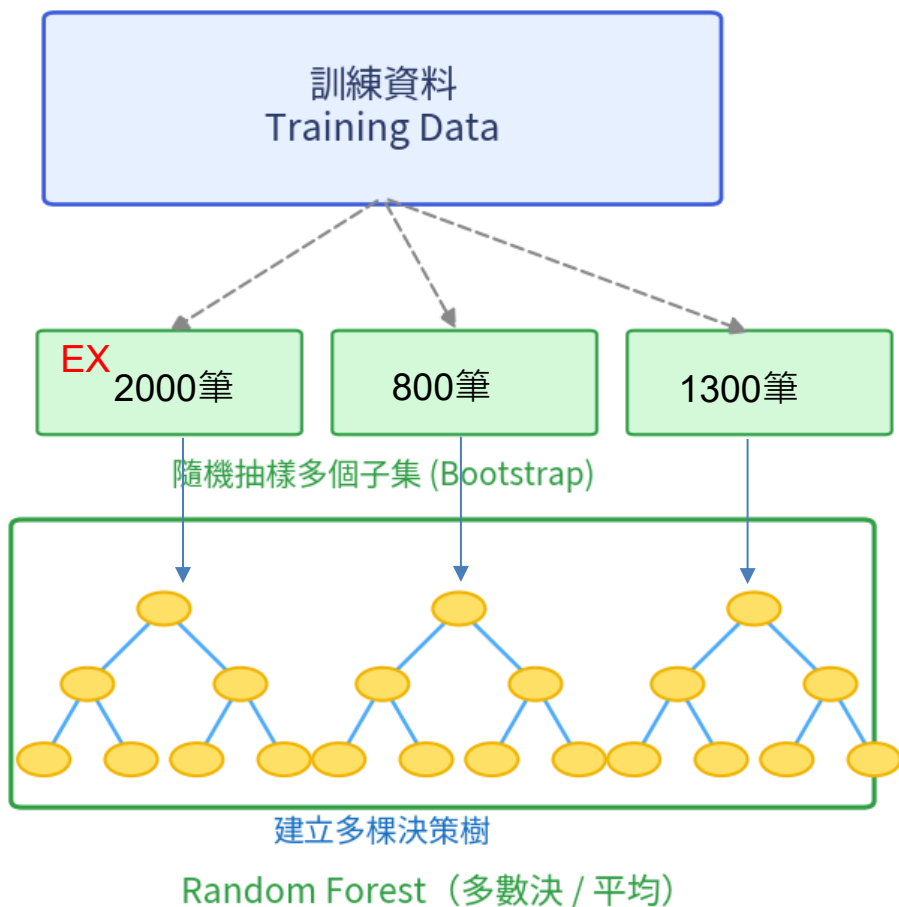


Gradient Boosting

沿損失函數梯度方向逐步擬合殘差，降低偏差。

Random Forest : Bagging 集成學習

從訓練資料隨機抽取多個子集，各自建立決策樹，再整合結果



Bagging (裝袋法)

對訓練資料做隨機抽樣(Bootstrap)，產生多個子集，各自訓練一棵樹。

降低變異數

多棵獨立的樹取平均(或多數決)，抵消個別樹的過擬合，模型更穩定。

整合決策

分類用多數決、迴歸取平均，結果優於單一決策樹。

本研究角色

作為機器學習(ML)代表模型，與統計、深度學習、LLM 比較。

決策樹 vs 隨機森林對照表

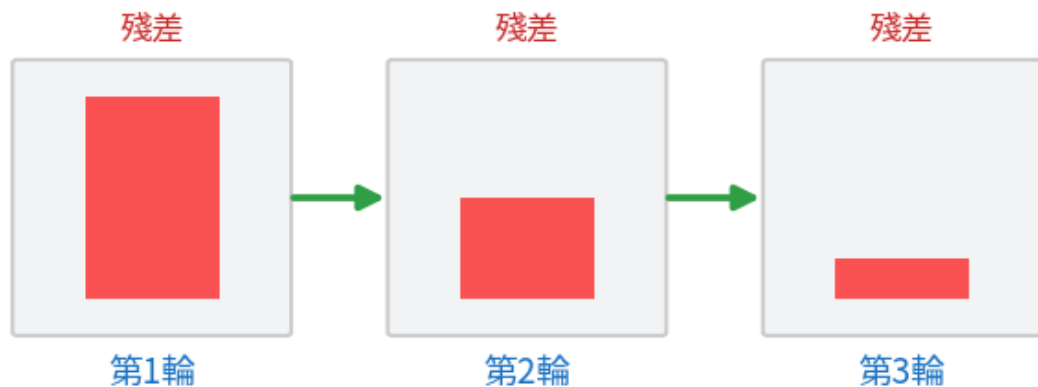
比較項目	決策樹	隨機森林
模型性質	單一模型	多模型集成
穩定性	容易過擬合	較穩定
訓練方式	單棵樹	多棵樹隨機訓練
結構透明度	高	較低
準確度	中等	通常較高
計算成本	低	高(需訓練多棵樹)

● SPY (樣本 2464)			
模型	訓練 MSE	測試 MSE	測試 MAPE(%)
Decision Tree	2.0362	2782.10	8.64
Random Forest	2.4263	2725.00	8.44
XGBoost	5.3500	2797.95	8.66
AdaBoost	20.5274	3398.31	9.80
Gradient Boosting	4.3163	2821.29	8.66
● QQQ (樣本 2464)			
模型	訓練 MSE	測試 MSE	測試 MAPE(%)
Decision Tree	1.9194	3362.08	11.63
Random Forest	2.3133	3137.76	11.09
XGBoost	5.3130	3291.94	11.18
AdaBoost	17.3061	3596.48	11.88
Gradient Boosting	4.2605	3277.47	11.40
● 0050.TW (樣本 2382)			
模型	訓練 MSE	測試 MSE	測試 MAPE(%)
Decision Tree	0.0118	45.87	9.76
Random Forest	0.0136	35.11	8.63
XGBoost	0.0299	37.17	8.91
AdaBoost	0.1067	33.98	8.52
Gradient Boosting	0.0233	35.67	8.90

實測：決策樹「訓練誤差最低、測試誤差最大」= 過擬合；
隨機森林泛化更佳，印證上表準確度差異。

Gradient Boosting (梯度提升)

沿損失函數梯度方向，逐步擬合殘差以降低偏差



每一輪把剩下的誤差(殘差)再擬合一次 → 越來越小

粗略預測

算出「實際值 - 預測值」= 殘差 (還差多少)

擬合殘差

每個新模型專門預測前面模型剩下的誤差(殘差)。

梯度下降

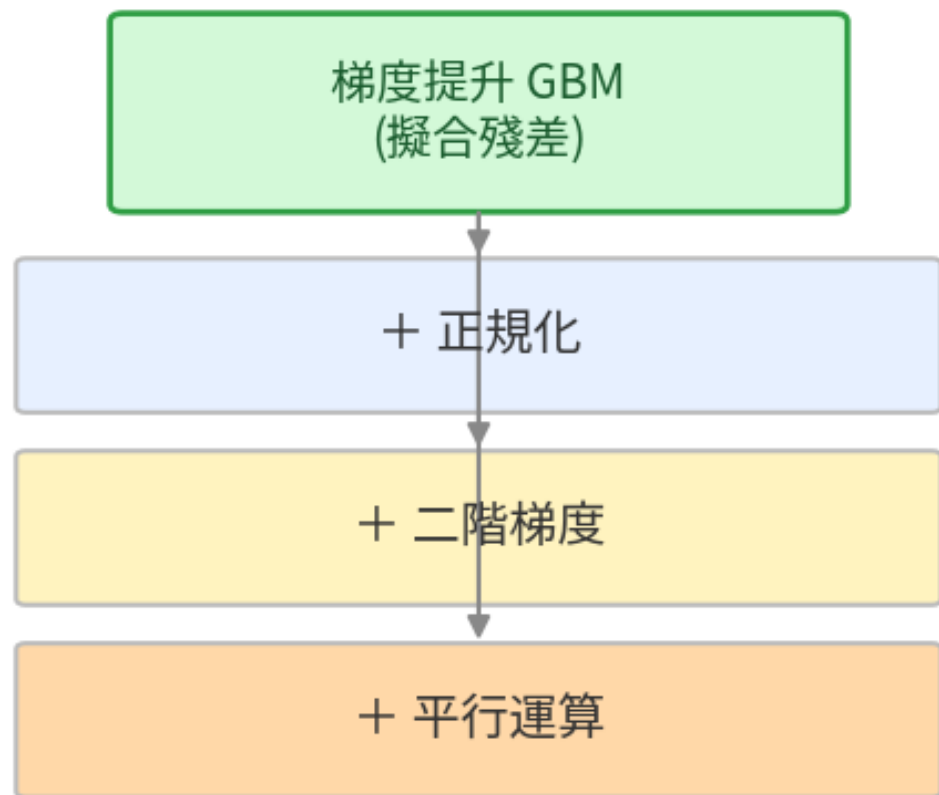
沿損失函數梯度方向逐步優化，降低偏差。

逐步降低偏差

弱學習器累加，整體誤差越來越小。

XGBoost (極限梯度提升)

Boosting 經典模型，含正規化抑制過擬合，速度快、精度高



以 **GBM** 為基礎

同樣用 Boosting 擬合殘差，但做了多項工程強化。

正規化抑制過擬合

目標函數加入正規化項(L1/L2)，控制複雜度。

速度與精度

採二階梯度近似與平行運算，訓練快、預測準。

★ 本研究角色

機器學習(ML)集成模型之一，與 RF/AdaBoost/GBM 比較。

AdaBoost (適應性提升)

逐輪提高「被分錯樣本」的權重，讓下一輪更專注難分的樣本

第1輪



第2輪



第3輪



紅色=被分錯、權重變大 藍色=正確

弱學習器

用全部資料訓練一棵很淺的樹

樣本加權

每一輪結束後，提高「被分錯的樣本」權重，下一輪更專注它們。

串聯弱學習器

串聯多個簡單模型(通常是淺決策樹)，逐步補強。

加權

最終依各學習器表現加權整合結果。

XGBoost、AdaBoost、Gradient Boosting 比較

比較項目	AdaBoost	Gradient Boosting	XGBoost
核心機制	逐輪提高被分錯樣本的權重，串聯弱學習器	沿損失函數梯度逐步擬合前一輪殘差	以 GBM 為基礎，做多項工程強化
防過擬合	弱學習器(淺樹)串聯逐步補強	以學習率縮小每步更新幅度	L1/L2 正規化 + 學習率 0.1
運算效率	序列訓練、較慢	序列訓練、較慢	二階梯度近似 + 平行運算，最快
對離群值	較敏感（會加大其權重）	相對穩健	相對穩健
本研究 SPY MSE	3398.31	2821.29	2797.95

三者皆 Boosting,XGBoost 最完整。但本研究中三者迴歸誤差接近,且都明顯高於 ARIMA 與 LSTM。共同原因:底層皆為決策樹,有外推限制(詳見後續結果)。

樹模型：四種集成模型核心程式碼

sklearn.ensemble + xgboost · TimeSeriesSplit 驗證 (run_regression.py)

參數說明

sklearn.ensemble：集成模型模組，隨機森林以多棵決策樹平均、抑制過擬合。

n_estimators=100 樹的數量

| max_depth 樹深(RF 10 / XGB·GBM 4) |

learning_rate=0.1 步長縮減防過擬合 |

random_state=42 固定種子可重現

```
12 from sklearn.ensemble import RandomForestRegressor, AdaBoostRegressor, GradientBoostingRegressor
13 from sklearn.model_selection import TimeSeriesSplit
14 from sklearn.metrics import mean_squared_error
15 from xgboost import XGBRegressor
16 warnings.filterwarnings("ignore")
```

```
49 def models():
50     return {
51         "Random Forest": RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42),
52         "XGBoost": XGBRegressor(n_estimators=100, max_depth=4, learning_rate=0.1, random_state=42, verbosity=0),
53         "AdaBoost": AdaBoostRegressor(n_estimators=100, random_state=42),
54         "Gradient Boosting": GradientBoostingRegressor(n_estimators=100, max_depth=4, random_state=42),
55     }
```

說明：

- RF / AdaBoost / GBM 來自 sklearn.ensemble，XGBoost 來自 xgboost 套件。
- 四個模型皆以 TimeSeriesSplit (時序交叉驗證) 評估，依時間切分、訓練集永遠早於驗證集，避免用未來資料預測過去 (資料洩漏) 。

循環神經網路 (Recurrent Neural Network, RNN)

處理序列資料的深度學習模型，是 LSTM 的前身

RNN

專門處理序列資料(Sequential Data)的深度學習模型。

循環連結 = 記憶

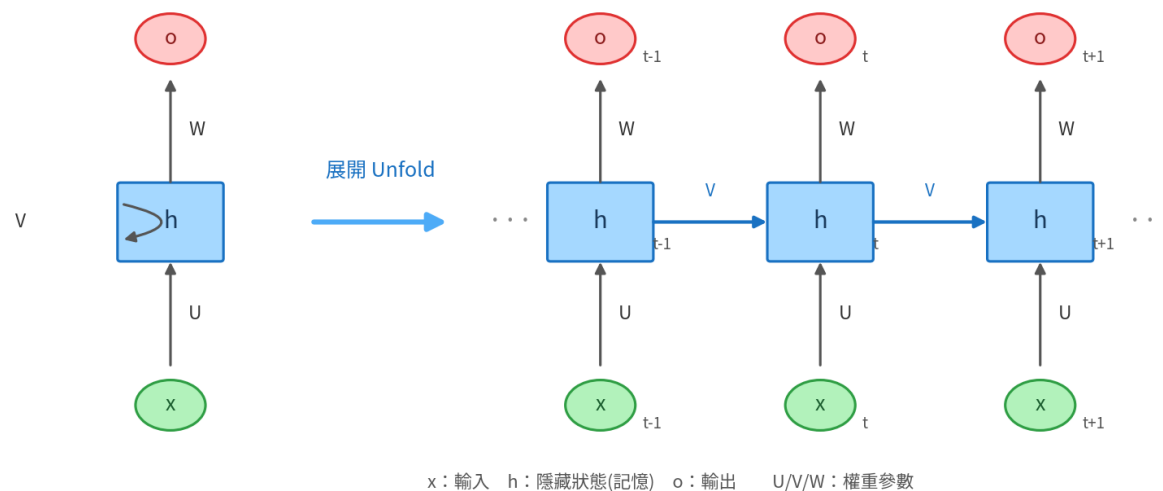
神經元之間存在「循環連結」，能保留過去資訊的「記憶」，這是與前饋神經網路最大的不同。

訓練方式

透過時間反向傳播(BPTT)訓練,需沿時間逐步相乘梯度。

長期依賴問題

序列一長,梯度連乘趨近於零(梯度消失),難記住早期資訊 → 因此發展出 LSTM、GRU。



模型演進

RNN → LSTM → GRU

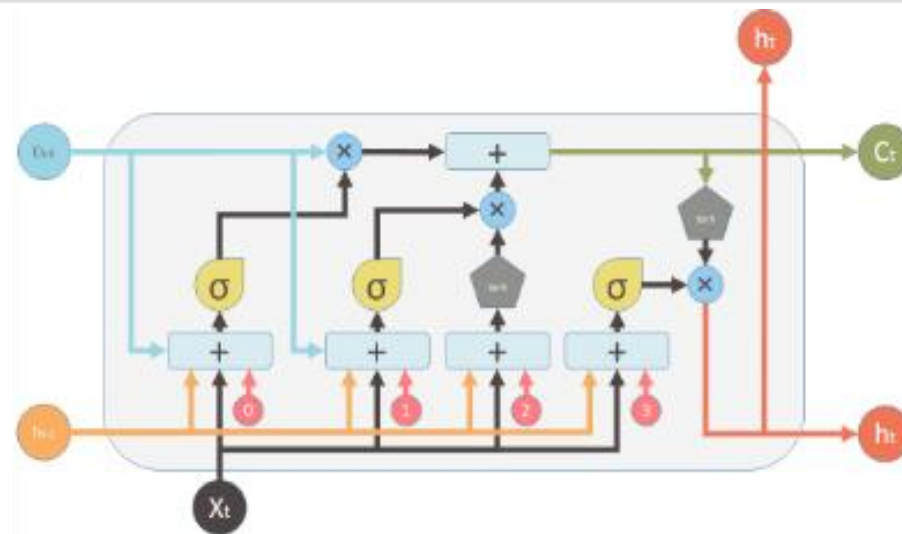
長短期記憶網路（Long Short-Term Memory, LSTM）

透過細胞狀態與三個閘門，記住長期依賴、解決傳統 RNN 的梯度消失問題

核心結構：細胞狀態（Cell State）

- LSTM 的關鍵在於細胞狀態，可想像成一條貫穿所有單元的「傳送帶」，資訊在上面流動而保持不變。LSTM 透過三個閘門精細控制這條傳送帶上的資訊增減。
- **遺忘閘門**（Forget Gate）：用 Sigmoid 決定丟棄哪些
- **輸入閘門**（Input Gate）：決定哪些新資訊要寫入
- **輸出閘門**（Output Gate）：依細胞狀態決定輸出，成為下一時點的隱藏狀態 h_t

股價為典型時間序列，LSTM 能捕捉長短期價格依賴，且端到端學習、不需手動特徵工程。



LSTM 結構圖 — 符號對照

輸入 (Inputs)

- x_t 輸入向量
- C_{t-1} 前一區塊的記憶
- h_{t-1} 前一區塊的輸出

偏置 (Bias)

- 0 每個閘門的偏置項 (0,1,2,3)

輸出 (Outputs)

- C_t 當前區塊的記憶
- h_t 當前區塊的輸出

非線性 (Nonlinearities)

- σ Sigmoid (壓縮到 0~1, 當閘門)
- $\tanh$ 雙曲正切 tanh (壓縮到 -1~1)

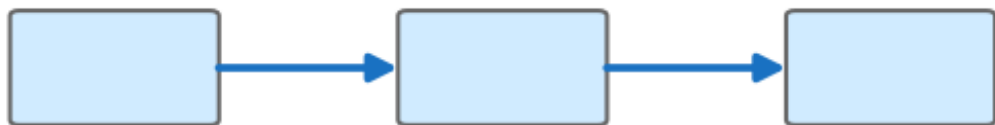
向量運算 (Vector operations)

- $\times$ 逐元素相乘 (element-wise $\times$)
- $+$ 相加 / 串接 (+)

各種 LSTM：單向x雙向x單層x雙層

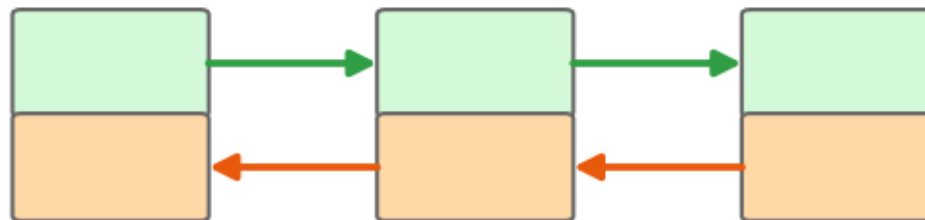
依資訊流動方向分單向與雙向,再依網路深度分單層與雙層,本研究共四種變體。

單向 LSTM (Unidirectional)



只看過去 → 現在 (順時間)

雙向 LSTM (Bidirectional)



同時看過去與未來 (雙向)

單向 LSTM

只用「過去」資訊預測未來，
符合真實預測情境（不偷看未來）。

本研究分為單層、雙層兩種。

雙向 LSTM (BiLSTM)

同時讀取前後文資訊。用於預測「未來」時等於偷看未來，
造成資訊洩漏——這正是本研究中 BiLSTM 表現最差的原因。

本研究同樣實作單層、雙層兩種

深度學習：LSTM 核心程式碼

Keras Sequential · Dense(1)+mse 做迴歸 (models.py) TensorFlow

參數說明

LSTM(50)：50 個神經元（隱藏狀態維度）。
return_sequences=True：回傳每個時間步輸出，供下一層 LSTM 使用（疊雙層時第一層需設）。
Dense(1)：輸出 1 個數值（次日收盤價）。
optimizer='adam'：Adam 最佳化器，收斂穩定。
loss='mse'：均方誤差；
 用 mse + 無 sigmoid = 迴歸版本。

```

6  from tensorflow.keras.models import Sequential
7  from tensorflow.keras.layers import LSTM, Dense, Bidirectional, Input

85 # ===== 四種 LSTM 變體(迴歸, 新增) ===== 【新增】
86 class SingleLayerLSTMRegressor(DeepLearningRegressor):
87     def __init__(self, input_shape):
88         self.model = Sequential([Input(shape=input_shape), LSTM(50), Dense(1)])
89         self.model.compile(optimizer='adam', loss='mse')
90
91 class DoubleLayerLSTMRegressor(DeepLearningRegressor):
92     def __init__(self, input_shape):
93         self.model = Sequential([Input(shape=input_shape),
94                                 LSTM(50, return_sequences=True), LSTM(50), Dense(1)])
95         self.model.compile(optimizer='adam', loss='mse')

```

說明：

- 單層 LSTM：一層 LSTM(50) + Dense(1) 輸出數值。
- 雙層 LSTM：疊兩層 LSTM（第一層 return_sequences=True）。
- 關鍵：Dense(1) 無 sigmoid、loss='mse'，即為迴歸版本。

深度學習：雙向 LSTM (BiLSTM) 核心程式碼

Keras Sequential · Bidirectional 包住 LSTM 做迴歸 (models.py)

```
91 class DoubleLayerLSTMRegressor(DeepLearningRegressor):
92     def __init__(self, input_shape):
93         self.model = Sequential([Input(shape=input_shape),
94                                   LSTM(50, return_sequences=True), LSTM(50), Dense(1)])
95         self.model.compile(optimizer='adam', loss='mse')
96
97 class SingleLayerBiLSTMRegressor(DeepLearningRegressor):
98     def __init__(self, input_shape):
99         self.model = Sequential([Input(shape=input_shape), Bidirectional(LSTM(50)), Dense(1)])
100        self.model.compile(optimizer='adam', loss='mse')
101
102 class DoubleLayerBiLSTMRegressor(DeepLearningRegressor):
103     def __init__(self, input_shape):
104         self.model = Sequential([Input(shape=input_shape),
105                                   Bidirectional(LSTM(50, return_sequences=True)),
106                                   Bidirectional(LSTM(50)), Dense(1)])
107        self.model.compile(optimizer='adam', loss='mse')
```

說明：

- Bidirectional：同時正向 + 反向讀序列再合併，擷取更完整前後文。
- 單層 = Bidirectional(LSTM(50))；雙層 = 第一層 return_sequences=True。
- adam / mse / Dense(1) 無 sigmoid = 迴歸版本。

LLM 分類任務：GPT-3.5-turbo（預測次日漲跌）

以提示詞讓 GPT-3.5 預測「上漲 / 下跌」，SPY 共 200 筆

0.524

F1-score

0.51

準確率
Accuracy

0.454

精確率
Precision

0.621

召回率
Recall

結果接近隨機

F1 約 0.52；RF/LSTM 雖達 0.69~0.73，但同樣未超越持續看漲 (Always-Up 基準) 0.736，方向預測皆無實質效果。

分類非 LLM 強項

GPT-3.5 以文字推理漲跌方向，傾向一致猜「漲」，難以穩定判別次日方向。

對比：LLM 迴歸表現佳

同一 LLM 改做數值迴歸（GPT-4o-mini），三市場 R^2 一致達 0.80 顯示 LLM 較擅長「數值預測」而非「漲跌分類」。

LLM 迴歸任務：GPT-4o-mini（預測次日收盤價）

以 60 天技術指標讓 GPT-4o-mini 預測次日收盤價，SPY 共 50 筆 (llm_regression.py)

```
===== LLM 迴歸預測結果 =====
有效樣本數：50 / 50
均方誤差 (MSE): 0.23
均方根誤差 (RMSE): 0.48
平均絕對誤差 (MAE): 0.37
決定係數 (R²): 0.7845
詳細結果已儲存至 llm_regression_0050_tw.csv
```

設定：模型 gpt-4o-mini，temperature=0.7；
輸入過去 60 天 5 指標（Close、MA20、MA60、RSI、波動率），
輸出 JSON 次日收盤價；測試集 2022 年後各 50 筆。
觀察：三市場 R^2 一致落在 0.80 左右（0.784~0.815），顯示 LLM
具穩定的數值預測能力。
⚠ **MSE/RMSE** 不可跨市場比（受股價尺度影響：0050 約 50 元、
SPY/QQQ 數百元）；跨市場要看 **R^2** （標準化、可比）。

```
===== LLM 迴歸預測結果 =====
有效樣本數：50 / 50
均方誤差 (MSE): 41.40
均方根誤差 (RMSE): 6.43
平均絕對誤差 (MAE): 5.25
決定係數 (R²): 0.8082
詳細結果已儲存至 llm_regression_spy.csv
```

```
===== LLM 迴歸預測結果 =====
有效樣本數：50 / 50
均方誤差 (MSE): 61.61
均方根誤差 (RMSE): 7.85
平均絕對誤差 (MAE): 6.50
決定係數 (R²): 0.8159
詳細結果已儲存至 llm_regression_qqq.csv
```


大語言模型：GPT-4o-mini 核心程式碼

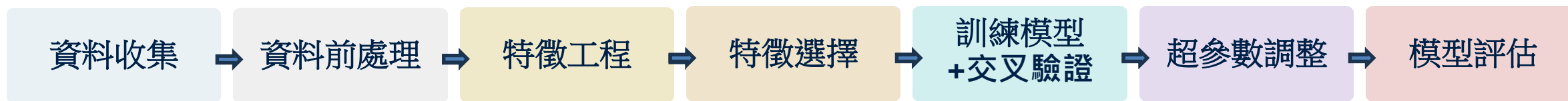
OpenAI API · temperature=0.7 (llm_regression.py)

```
try:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.7,
        response_format={"type": "json_object"}
    )
    content = response.choices[0].message.content
    data = json.loads(content)
    pred_price = data.get("predicted_close")
```

說明：

- temperature=0.7：控制輸出隨機性（範圍 0~2）。
- 雖然 0.7 會帶來一點隨機性，但我重跑後三個市場的 R^2 仍然維持在 0.8 左右，代表 LLM 的數值預測能力是穩定的。

機器學習的標準流程 (Machine Learning Pipeline)



1. 資料收集

蒐集原始資料(資料庫、API 等)，資料品質直接影響表現。

2. 資料前處理

清理資料、處理缺失值；特徵工程(標準化、編碼)；切分訓練/驗證/測試集。

3. 模型選擇

分類: 邏輯迴歸、決策樹、隨機森林、SVM、神經網路。
迴歸: 線性迴歸、Ridge、Lasso、XGBoost。

4. 模型訓練

用訓練集學習，調整超參數，並以交叉驗證(Cross-Validation)評估穩定度。

5. 模型評估

分類: Accuracy、Precision、Recall、F1-score、ROC-AUC。
迴歸: MSE、RMSE、MAE、 R^2 、MAPE。

6. 模型調整

以超參數調整(Grid Search、貝葉斯優化)或特徵工程改進效能。

7. 部署與監控

部署至實際系統(API、雲端)；持續監控、避免概念漂移(Concept Drift)。

本研究實驗流程：對應 ML Pipeline 的實作

資料來源

Yahoo Finance API
(update_database.py)

特徵工程

Lag/MA/Vol

性能評估

MSE/MAPE/F1

資料前處理

清洗、缺失值處理
→ 存入 etf_data.db

模型訓練

RF / LSTM / ARIMA:
TimeSeriesSplit(5 折)
ARIMA 於每折內採 1 步 rolling

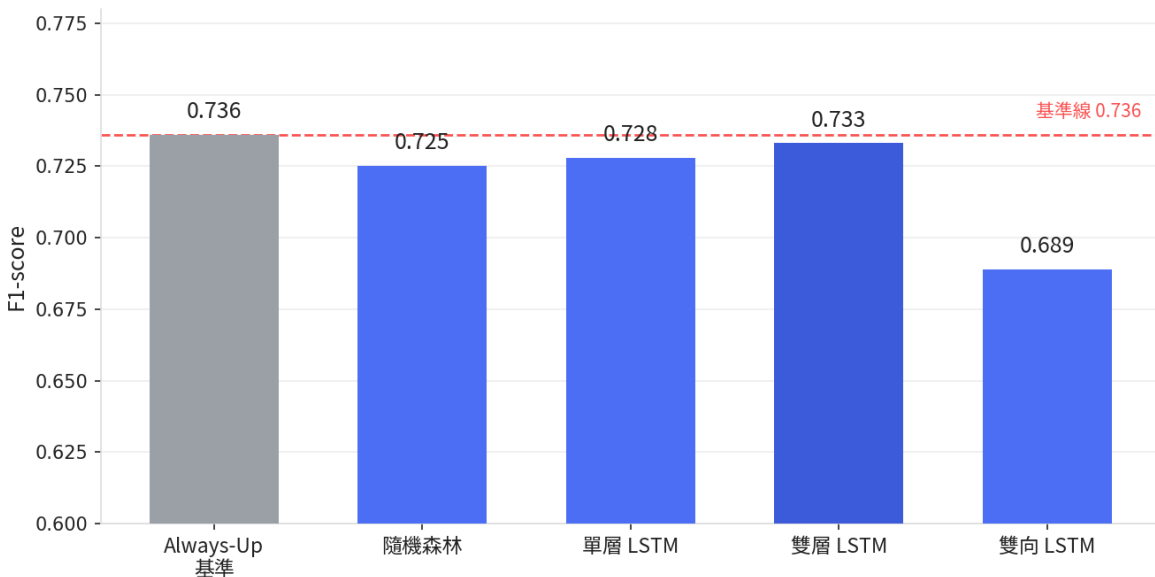
實驗設計：資料集與特徵

類型	細節描述
預測標的	SPY (美股大盤), QQQ (科技股), 0050.TW (台股)
資料期間	2016 – 2026 (日級別資料)
原始特徵	OHLCV (開盤價、最高價、最低價、收盤價、成交量)
技術指標	MA20, MA60, RSI(14), 年化波動率(20)
交叉驗證	TimeSeriesSplit 時間序列交叉驗證，確保訓練集早於測試集

迴歸另採落後價 (Lag1/2/5) 與 MA10/MA50、波動率特徵

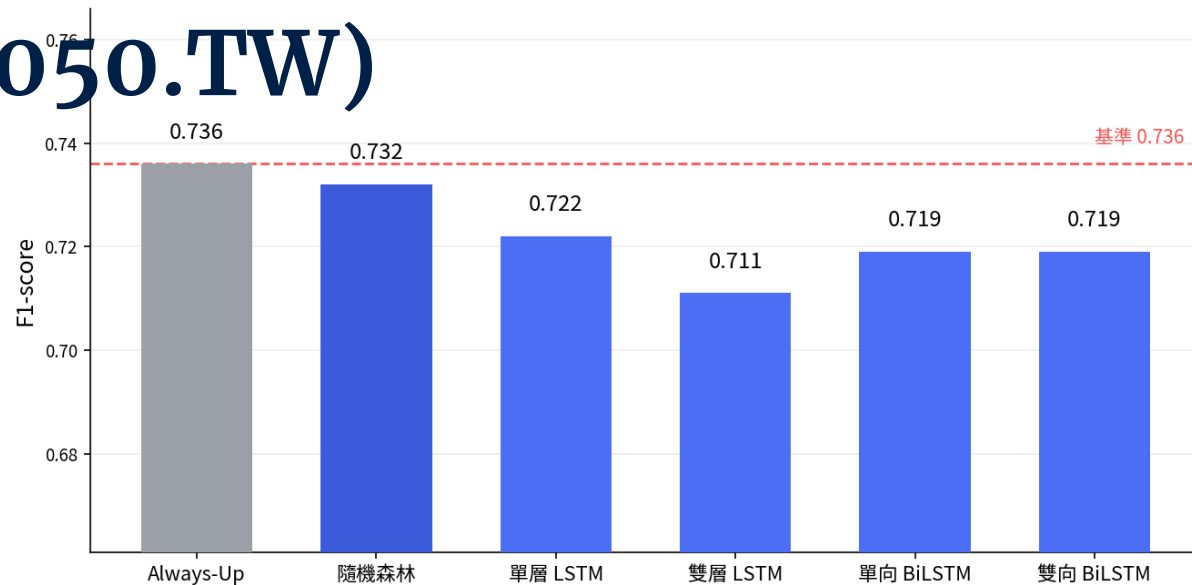
分類問題結果 (SPY/QQQ/0050.TW)

SPY 各模型分類 F1-score 比較 (含 Always-Up 基準)

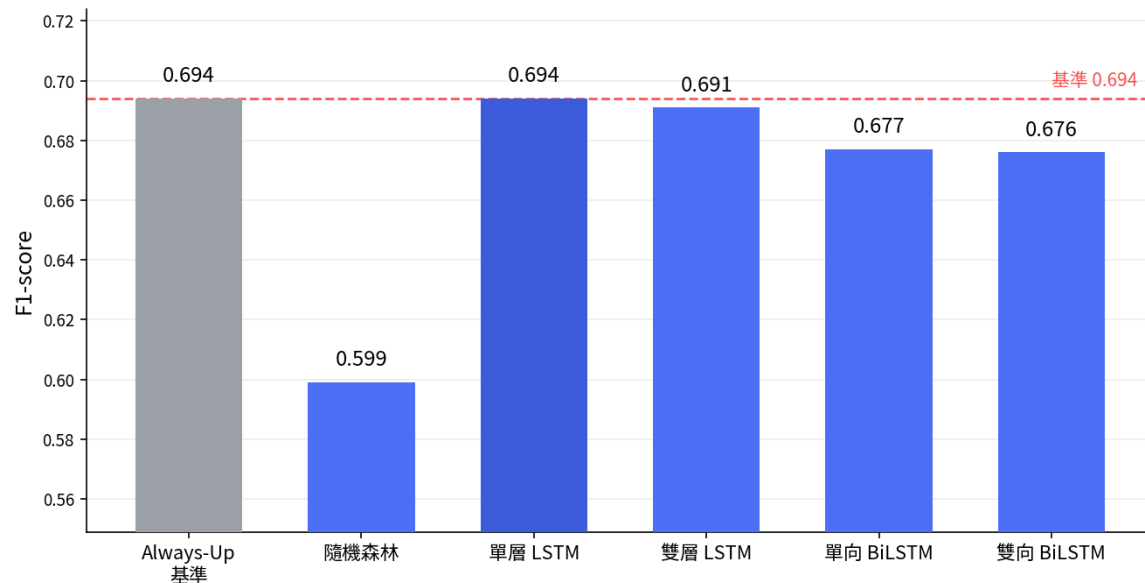


- 四個模型 F1 均貼近或低於基準 (0.736)：高分主要來自多頭市場的**上漲偏態**，而非具備方向判斷力；雙向 LSTM 最差。
- 隨機森林在 0050 僅 0.599 (低於基準)：LSTM 數據上較貼近基準，而隨機森林嘗試以特徵判斷方向、偶爾預測跌，在 0050 的表現反而低於單純猜漲。
- 核心結論**：多頭偏態下，分類任務的 F1 指標鑑別力有限。

QQQ 各模型分類 F1-score 比較 (含 Always-Up 基準)



0050.TW 各模型分類 F1-score 比較 (含 Always-Up 基準)



迴歸結果：各模型誤差比較（MSE / MAPE）

模型	SPY 測試 MSE	QQQ 測試 MSE	0050 測試 MSE
ARIMA	19.57	21.53	0.18
Random Forest	2725.00	3137.76	35.11
XGBoost	2797.95	3291.94	37.17
AdaBoost	3398.31	3596.48	33.98
Gradient Boosting	2821.29	3277.47	35.67
單層 LSTM	125.56	200.68	1.77
雙層 LSTM	132.77	279.36	1.62

模型	SPY MAPE	QQQ MAPE	0050 MAPE
ARIMA	0.79%	1.04%	0.87%
Random Forest	8.44%	11.09%	8.63%
XGBoost	8.66%	11.18%	8.91%
AdaBoost	9.80%	11.88%	8.52%
Gradient Boosting	8.66%	11.40%	8.90%
單層 LSTM	2.14%	3.36%	2.94%
雙層 LSTM	2.41%	3.73%	3.02%

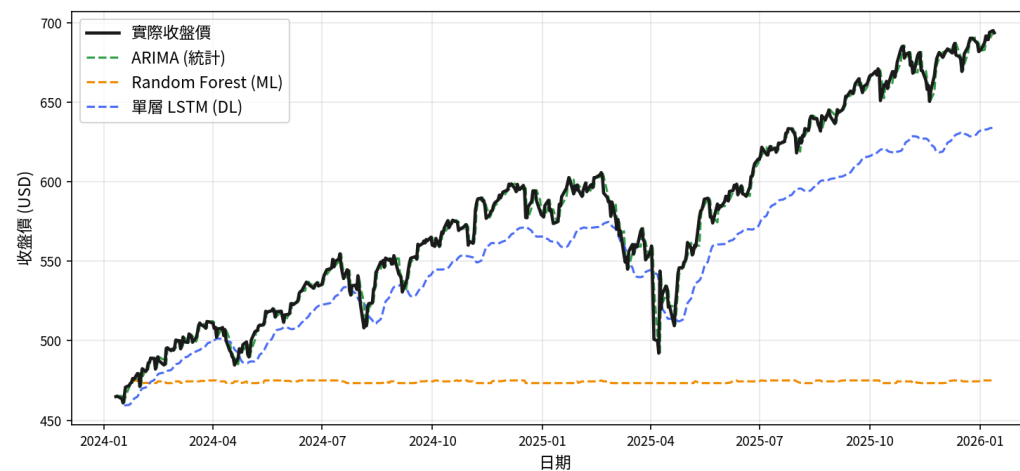
※ **MSE** 受股價尺度影響，僅供同市場內比較；**MAPE** 為百分比，可跨市場比較。
※ 同市場內排序一致：**ARIMA** < **LSTM** < 集成模型（誤差越小越佳）。

迴歸結果:預測曲線比較

QQQ 次日收盤價：實際 vs 各模型預測



SPY 次日收盤價：實際 vs 各模型預測



0050.TW 次日收盤價：實際 vs 各模型預測



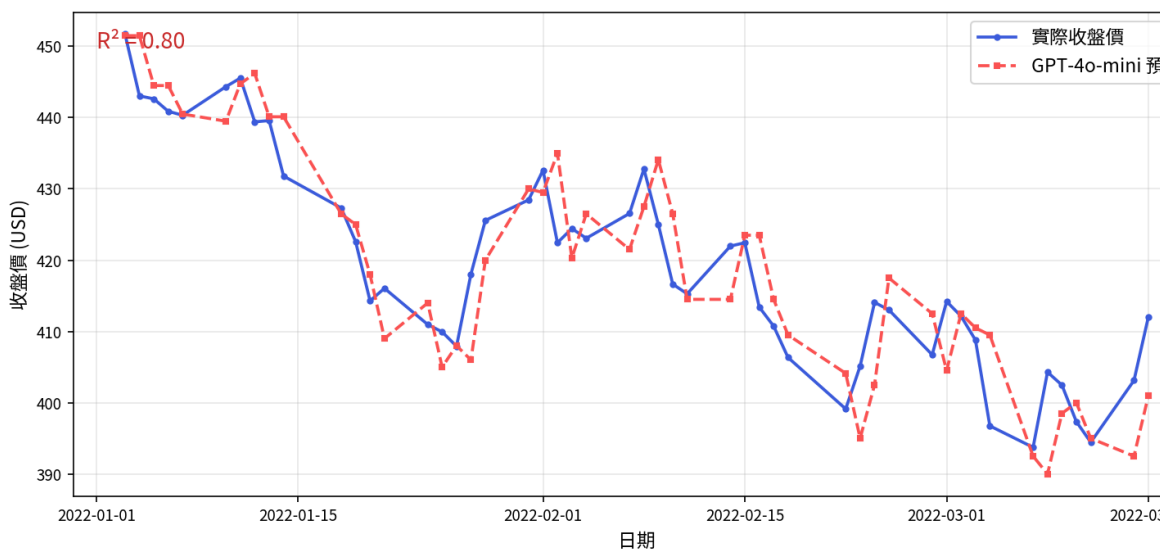
- 預測曲線對照(**SPY** 測試期)
- 綠線 **ARIMA**:幾乎貼合實際
- 橘線 隨機森林:無法外推、預測躺平
- 藍線 **LSTM**:能跟上趨勢、略有落差

迴歸結果：GPT-4o-mini效能(SPY/QQQ)

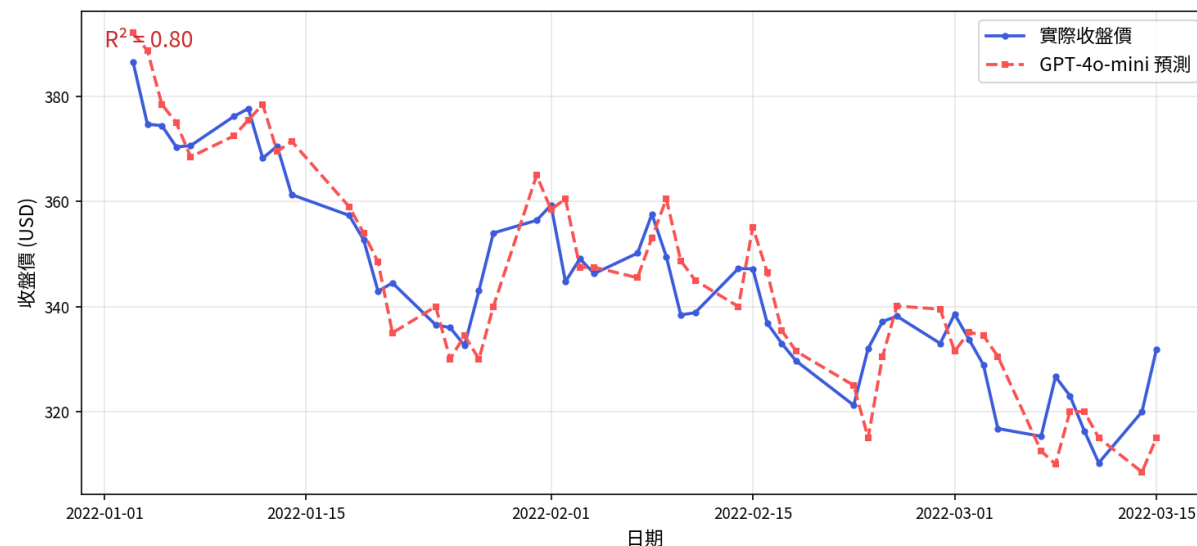
市場	R ²	MSE	RMSE	MAE
SPY	0.800	43.23	6.58	5.23
QQQ	0.802	66.11	8.13	6.62
0050	0.802	0.21	0.46	0.35

GPT-4o-mini 在 SPY、QQQ 各 50 個交易日的小樣本上預測次日收盤價，R² 皆達 0.80。藍線為實際、紅線為預測，趨勢貼合度高。

GPT-4o-mini 次日收盤價預測 — SPY (小樣本 50 個交易日)

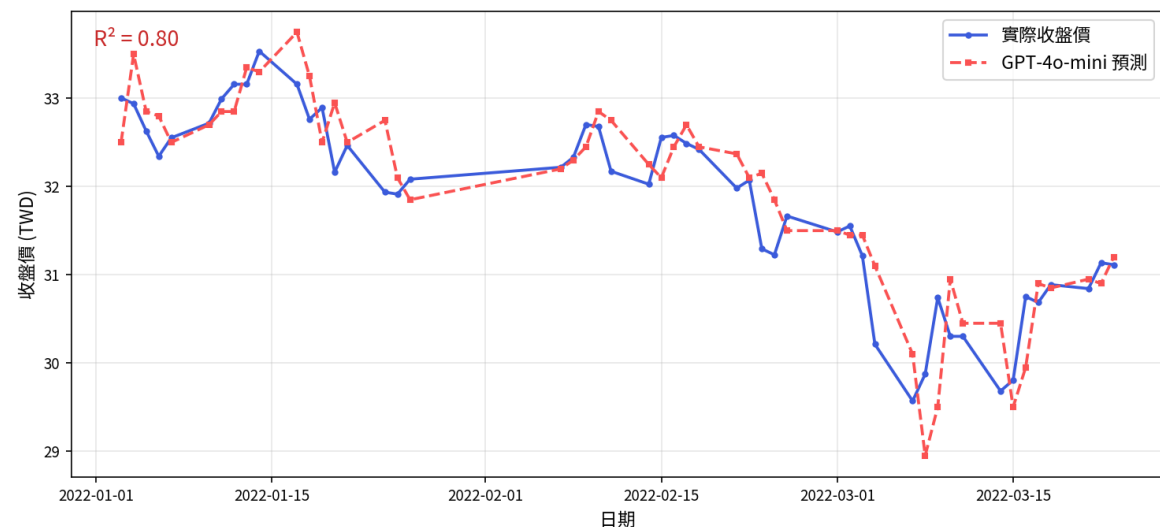


GPT-4o-mini 次日收盤價預測 — QQQ (小樣本 50 個交易日)



迴歸任務小結

GPT-4o-mini 次日收盤價預測 — 0050.TW (小樣本 50 個交易日)



一、模型排序一致：ARIMA < LSTM < 集成模型

以 MAPE (跨市場可比) 衡量，三個市場排序一致：

ARIMA (0.7~1.1%) 優於 LSTM (2~4%) 優於樹模型 (8~12%)。顯示在價格預測上，較簡單的統計模型反而比複雜模型穩定。

二、集成模型的限制：無法外推

集成模型(RF/XGBoost等)無法預測超出訓練範圍的新高價，在長期上漲的測試期誤差最大，這是其結構限制而非調參問題。

三、LLM：三市場 R² 一致達 0.80

GPT-4o-mini 在小樣本(50天)下，SPY/QQQ/0050 的 R² 皆約 0.80，無需訓練、僅靠提示即可達此水準，顯示 LLM 在數值預測上的潛力。

→ 綜合而言：價格預測上「複雜模型不一定優於簡單模型」；
LLM 雖為小樣本探索，但跨市場一致性高，值得後續擴大驗證。

LLM-ETF 平台實作

- 技術架構：Streamlit + Plotly + SQLite
- 迴歸預測：預測次日報酬率，再換算回收盤價（避免外推問題）
- AI 輔助：整合 GPT 生成風險摘要

本平台整合迴歸預測與 LLM 風險摘要，圖表支援即時互動（縮放、查看數值、切換時間範圍）；介面輕量、一般設備即可執行，適合作為教學示範與輔助參考工具。



結論：分類與回歸比較

本研究兩種任務皆做，各回答不同問題

分類任務（預測漲跌方向）

- 涵蓋模型：機器學習（隨機森林）、深度學習（單/雙向 LSTM）、大語言模型（GPT-3.5）。
- 發現：F1 貼近基準,方向預測無實質效果

迴歸任務（預測收盤價）

- 涵蓋模型：統計（ARIMA）、機器學習（隨機森林/XGBoost/AdaBoost/GBM）、深度學習（單/雙層 LSTM）、大語言模型（GPT-4o-mini 零樣本預測）。
- 發現：排序一致,ARIMA 最佳

RQ1 效能:分類無冠軍(GPT-3.5 近亂猜);迴歸 ARIMA > LSTM > 樹;LLM 小樣本有潛力

RQ2 市場:台股 0050 比美股難預測

RQ3 指標:分類 F1 看不出差別,迴歸 MAPE 排序才清晰

RQ4 平台:並排比較+白話摘要

結論與未來研究方向

本研究貢獻

- 建立跨市場、跨世代比較基準
- 揭示集成無法外推的結構限制
- 實作開源 LLM 輔助平台

未來展望

- **盤中即時預測**:從日線收盤預測,延伸到盤中串流 (Kafka/Redis)。
- **擴大 LLM 樣本**:從每市場 50 筆擴至完整測試集
- **加入更多資訊**:導入新聞情緒與總體經濟指標
- **比較更多 LLM**:測試 Gemini、Llama、Gemma 等 平台
- **可用性評估**:以 SUS 量表做使用者實測

簡報結束，敬請指導